

How Companies Build Agents from Problem Statements

The actual step-by-step process that companies (and hackathon teams) follow to turn a problem into a working AI agent. This is the playbook for LaunchpadX.

The Golden Question

"What manual process can an AI do instead of a human?"

Every agent starts here. Not "what cool AI thing can we build?" but "what repetitive, rule-based, or knowledge-intensive task is a human doing right now that an AI could handle?"

The 7-Step Agent Building Process

Step 1: Decompose the Problem

Input: Problem statement

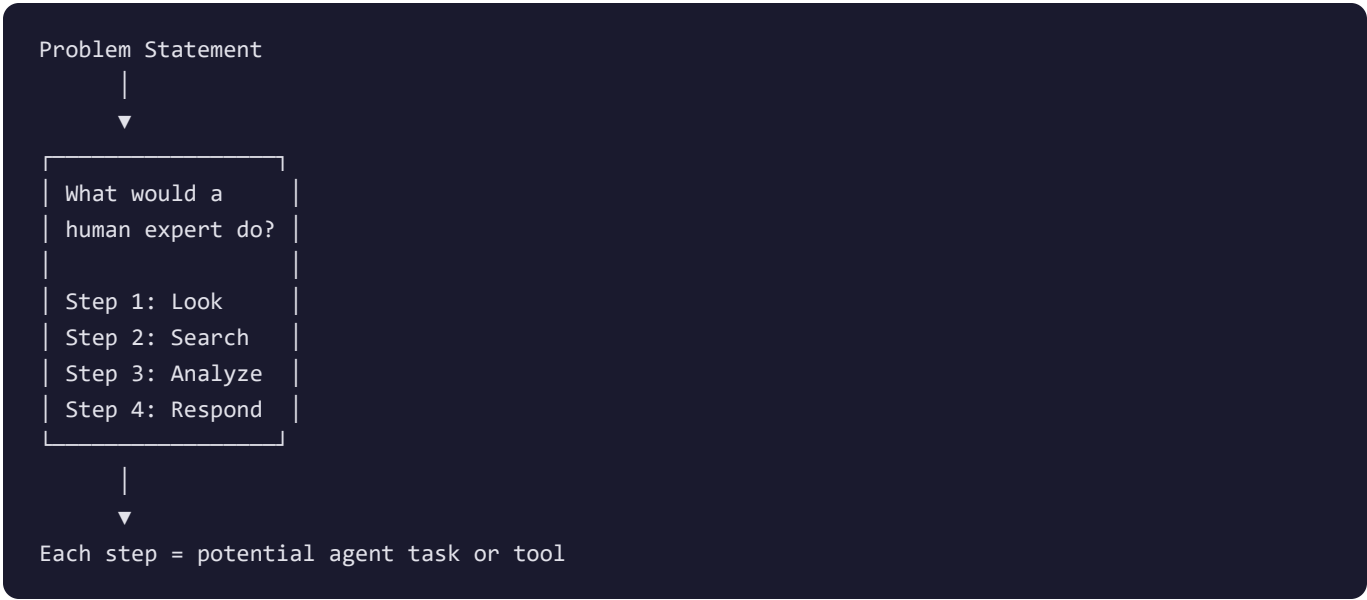
Output: List of tasks the agent needs to do

Example problem: *"Farmers in rural India struggle to identify crop diseases"*

Decompose into tasks:

1. Accept a photo of a crop
2. Identify the disease from the image
3. Search for treatment options
4. Recommend pesticides available locally
5. Provide instructions in the farmer's language

Key question: Can you describe what a human expert would do step-by-step? If yes, an agent can do it.



Step 2: Decide the Architecture

Based on the decomposition, choose:

If the problem...	Use this architecture
Has 1-3 simple steps	Single agent with tools
Has steps that need different expertise	Multi-agent pipeline (like our project)
Has steps that can run in parallel	Orchestrator-worker pattern
Needs to handle many similar requests differently	Router + specialist agents
Is conversational (back-and-forth)	Chatbot agent with memory

Quick decision for hackathon:

```
Can one LLM call with tools solve it?
├─ YES → Single agent (fastest to build, demo in 2 hours)
└─ NO →
    ├── Steps are sequential? → Pipeline (our style)
    ├── Steps are parallel? → Orchestrator-worker
    └─ Different user types? → Router + specialists
```

Step 3: Define the Tools

The agent is only as good as its tools. This is where most companies spend 70% of their time.

What the agent needs to do	Tool type	Example
Get external information	API call	Weather API, Google Maps, database query
Search the internet	Web search	Tavily, Serper, Google Search API
Read documents	RAG / retrieval	Vector store search, document parser
Process images	Vision model	GPT-4o vision, Claude vision
Generate files	File creation	python-pptx, docx, code generation
Send notifications	Messaging	Email, SMS, Slack, WhatsApp
Store/retrieve data	Database	SQLite, PostgreSQL, Airtable
Execute code	Sandbox	Code interpreter, Docker sandbox

For hackathon — keep tools simple:

- Web search (Tavily — 1 line to add)
- API calls (HTTP requests to free APIs)
- File generation (create downloadable output)
- Vision (if the problem involves images)

Step 4: Write the System Prompt

This is the agent's "brain" — its instructions, personality, and constraints.

The formula:

```
SYSTEM PROMPT = Role + Goal + Context + Rules + Output Format
```

Template:

You are a [ROLE] specializing in [DOMAIN].

Your goal is to [PRIMARY OBJECTIVE] by [HOW].

Context:

- You are helping [WHO]
- The user will provide [INPUT TYPE]
- You have access to these tools: [TOOL LIST]

Rules:

1. Always [IMPORTANT BEHAVIOR]
2. Never [DANGEROUS BEHAVIOR]
3. If unsure, [FALLBACK BEHAVIOR]

Output format:

Return your response as [FORMAT] with these fields:

- [FIELD 1]: [description]
- [FIELD 2]: [description]

Real example (crop disease agent):

You are an agricultural expert AI specializing in Indian crop diseases.

Your goal is to identify crop diseases from descriptions or images and recommend treatments using locally available products.

Context:

- You are helping farmers in rural India
- The user will provide a description of crop symptoms or a photo
- You have access to: web_search, crop_disease_database, local_pesticide_finder

Rules:

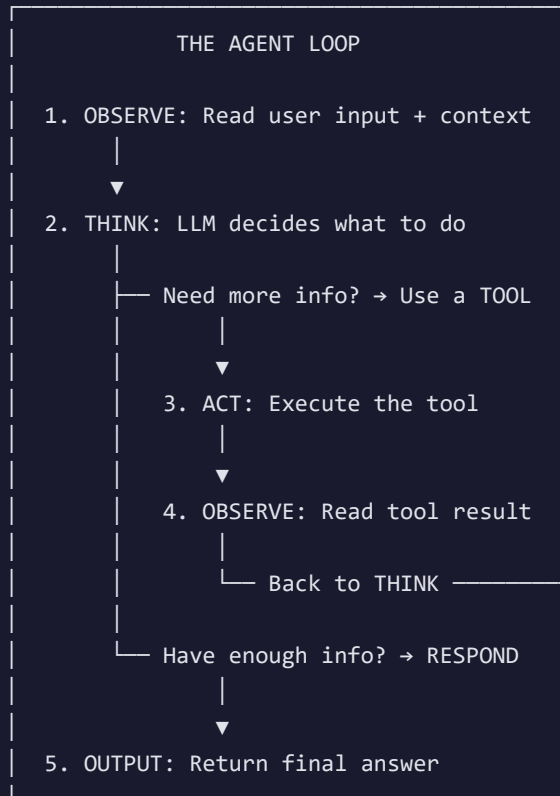
1. Always recommend treatments available in India
2. Never recommend banned pesticides
3. If unsure about diagnosis, say so and suggest consulting a local KVK
4. Provide response in simple language

Output format:

```
{
  "disease_name": "...",
  "confidence": "high/medium/low",
  "symptoms_matched": [...],
  "treatment": "...",
  "pesticides": [{"name": "...", "dosage": "...", "available_at": "..."}],
  "prevention": "..."
}
```

Step 5: Build the Agent Loop

The core logic that makes an agent an AGENT (not just a chatbot):



In code (simplified):

```
def agent_loop(user_input, tools, system_prompt, max_iterations=5):
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_input}
    ]

    for i in range(max_iterations):
        # THINK: Ask the LLM what to do
        response = llm.chat(messages, tools=tools)

        # Does it want to use a tool?
        if response.has_tool_calls:
            for tool_call in response.tool_calls:
                # ACT: Execute the tool
                result = execute_tool(tool_call.name, tool_call.args)
                # OBSERVE: Feed result back
                messages.append({"role": "tool", "content": result})
            else:
                # RESPOND: LLM is done, return the answer
                return response.content

    return "Max iterations reached"
```

This is the same pattern whether you use:

- Raw API calls (our approach)

- Claude Agent SDK (handles the loop for you)
- n8n (AI Agent node does this visually)
- LangGraph (graph-based loop)
- CrewAI (agent.execute does this)

Step 6: Add Memory & State

For agents that need to remember context:

Memory Type	What	When
Conversation memory	Chat history	Chatbots, support agents
Working memory	Current task state	Multi-step agents
Long-term memory	Past interactions	Personal assistants
Shared memory	State between agents	Multi-agent systems (our DB)

For hackathon: Simple conversation memory (just keep the message history) is enough. Don't over-engineer.

Step 7: Test, Fail, Fix, Repeat

The testing loop:

```
1. Give the agent a problem
2. Watch what it does (tool calls, reasoning)
3. Find where it fails:
  - Wrong tool chosen? → Fix system prompt
  - Bad tool output? → Fix the tool
  - Wrong reasoning? → Add examples to prompt
  - Too slow? → Use faster model or fewer steps
4. Try again
```

Common failures and fixes:

Failure	Fix
Agent doesn't use tools	Add explicit instructions: "You MUST search before answering"
Agent hallucinates	Add: "Only use information from tool results, never make up data"
Output format wrong	Add JSON examples in the prompt, use structured output
Agent loops forever	Add max iterations, add "if you have enough info, respond"
Too slow	Use smaller model, reduce tool calls, cache common queries

Real Company Examples

How Cognition Built Devin (Coding Agent)

Problem: "Developers spend too much time on routine coding tasks"

Decomposition:

1. Read the GitHub issue / task description
2. Understand the codebase (search files, read code)
3. Plan the changes needed
4. Write the code
5. Run tests
6. Fix failures
7. Open a pull request

Architecture: Single agent with many tools

Tools: File read/write, shell commands, browser, git

Prompt: "You are a software engineer..."

Loop: ReAct (reason about what to do, act, observe result)

Memory: Full codebase context + conversation history

How Sierra Built Customer Service Agent

Problem: "Customer service is expensive and slow"

Decomposition:

1. Understand the customer's question
2. Look up their account/order
3. Search knowledge base for answer
4. If simple → answer directly
5. If complex → escalate to human with context

Architecture: Router + specialist agents

Tools: CRM lookup, knowledge base search, order tracking API

Prompt: "You are a helpful customer service agent for [Brand]..."

Loop: Classify → Route → Resolve or Escalate

Memory: Customer history + conversation context

How Harvey Built Legal Agent

Problem: "Legal research takes hours of manual work"

Decomposition:

1. Understand the legal question
2. Search case law databases
3. Find relevant precedents
4. Analyze applicability to the current case
5. Generate a memo with citations

Architecture: Pipeline (research → analyze → write)

Tools: Legal database search, document analysis, citation checker

Prompt: "You are a senior legal research associate..."

Loop: Multi-step with human review

Memory: Case context + document library

The LaunchpadX Playbook

When You Get the Problem Statement

First 30 minutes:

1. Read the problem statement 3 times
2. Ask: "What would a human expert do to solve this?"
3. Write down 3-5 steps the human would take
4. Each step = an agent task or tool
5. Decide: single agent or multi-agent?

Next 60 minutes:

6. Pick your stack:
 - Fast prototype → n8n (visual, quick)
 - Full control → Raw API + our pipeline
 - Show off framework knowledge → Claude Agent SDK
7. Define 2-3 tools the agent needs
8. Write the system prompt
9. Build the agent loop
10. Get a basic version working

Remaining time:

11. Test with real inputs
12. Fix failures
13. Add polish (better output format, error handling)
14. Build the demo flow
15. Prepare the pitch

Pitch Structure for LaunchpadX

1. THE PROBLEM (30 sec)
"X is a problem because Y. Currently, humans spend Z hours doing it."
2. OUR AGENT (60 sec)
"We built an AI agent that [does the task]. Here's how it works:
 - It takes [input]
 - Agent [step 1], [step 2], [step 3]
 - Output: [tangible deliverable]"
3. LIVE DEMO (90 sec)
Show it working. Real input → real output.
4. THE TECH (60 sec)
"We used [architecture]. The agent has [N] tools.
It follows the [pattern] design pattern.
Built with [framework/stack]."
5. WHY IT MATTERS (30 sec)
"This saves [time/money]. Companies like [competitor] raised
\$[amount] solving a similar problem."

Quick-Start Templates

Template 1: Research Agent (Easiest)

```
# Takes a topic → searches web → writes a report
tools = [web_search, summarize]
prompt = "Research [topic] and write a comprehensive report"
output = "Markdown report with sources"
```

Build time: 1-2 hours

Template 2: Analysis Agent (Medium)

```
# Takes data → analyzes → provides insights + recommendations
tools = [data_loader, calculator, chart_generator]
prompt = "Analyze [data] and provide actionable insights"
output = "JSON with insights, charts, recommendations"
```

Build time: 2-4 hours

Template 3: Automation Agent (Medium)

```
# Takes a trigger → decides action → executes
tools = [api_call, email_sender, database_query]
prompt = "When [trigger], determine the best action and execute it"
output = "Action taken + confirmation"
```

Build time: 2-4 hours

Template 4: Multi-Agent Pipeline (Advanced — Our Style)

```
# Takes a problem → multiple agents collaborate → deliverables
agents = [planner, researcher, builder, presenter]
tools = [web_search, file_gen, pptx_gen]
prompt = "Each agent has a specialized role..."
output = "Complete project (code + docs + presentation)"
```

Build time: 4-8 hours (but we already have it built!)

The Secret Sauce (What Judges Love)

1. **Tangible output** — Don't just chat, PRODUCE something (file, report, dashboard)
2. **Live demo** — Show it working, not slides about how it works
3. **Real problem** — Solve something people actually deal with
4. **Tool use** — Agents that call APIs and search the web > agents that just talk
5. **Human-in-the-loop** — Show you thought about safety and control
6. **Architecture diagram** — One slide showing how agents connect
7. **Cost/speed numbers** — "Runs in 30 seconds, costs \$0.05 per request"

See How to Build AI Agents | Agent Design Patterns | Training Schedule - LaunchpadX | Our AI Software Company

agentic-ai #hackathon #launchpadx #building #knowledge
